Guia De Desenvolvimento
Rest
Spring MVC
Spring Boot
Git
2026-2
Parte 3 –
Persistência de Dados com Spring Data JPA

Introdução

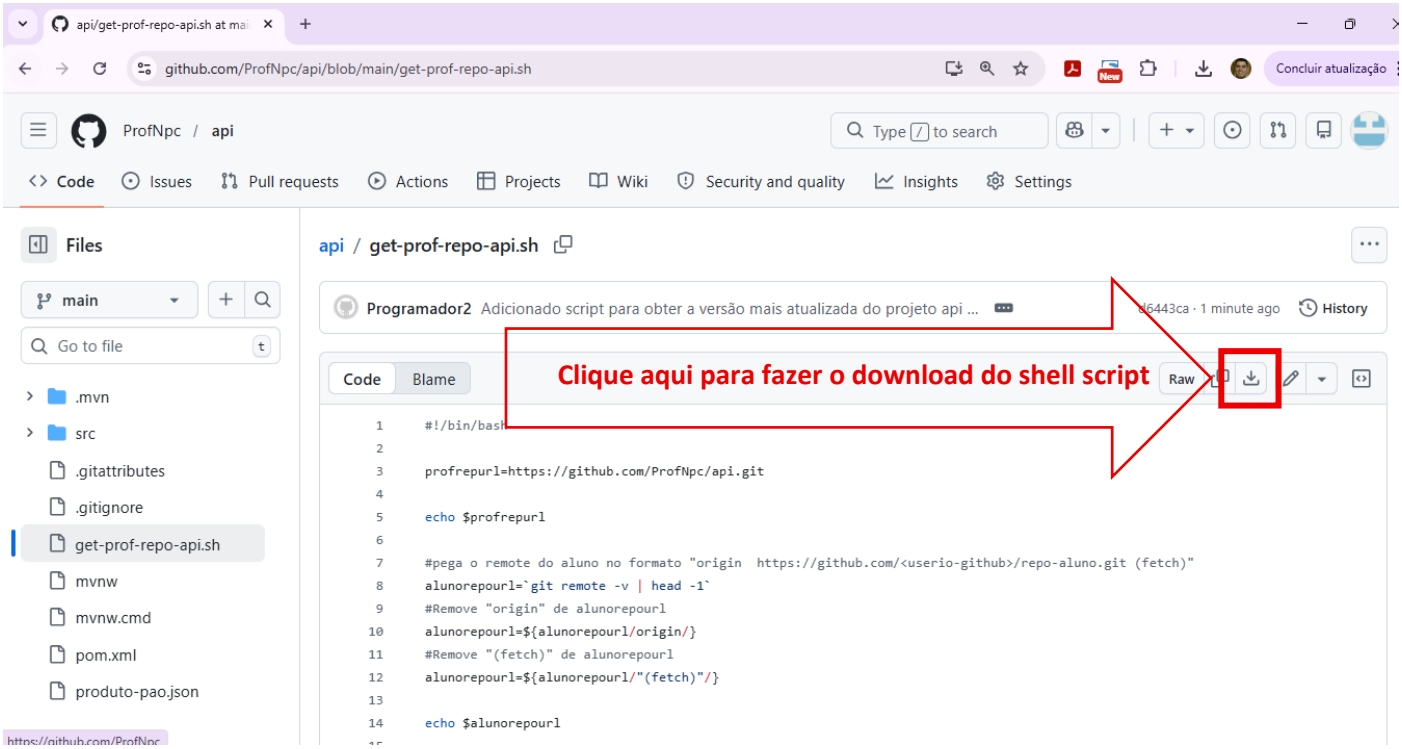
Neste documento teremos o passo-a-passo para a construção de uma aplicação web/rest utilizando as ferramentas Spring MVC, Spring Boot e Git.

Este guia se destina a dar suporte as aulas de LIP1 do segundo módulo do curso de informática noturno do ITB Belval, portanto, algumas explicações mais detalhadas serão omitidas pois serão apresentadas pelo professor durante a aula.

O projeto desenvolvido aqui funciona como referência para a criação do *BackEnd* dos projetos de TCC. A cada aula nós avançaremos com o desenvolvimento desse projeto, salvando seu progresso no repositório público https://github.com/ProfNpc/api.

Caso você venha a perder alguma aula, poderá obter a versão mais atualizada feita pelo professor e seguir acompanhando as aulas de onde a turma parou. Para isto basta colocar o *shell script* https://github.com/ProfNpc/api/blob/main/get-prof-repo-api.sh na raiz do projeto api e executá-lo em seguida.

Nesta parte iremos evoluir a aplicação para que ela deixe de trabalhar com coleções em memória e passe a persistir e recuperar dados de um banco de dados relacional, utilizando o padrão JPA (Java Persistence API) em conjunto com a biblioteca Spring Data JPA.



Parte 3

O que é JPA e como ele nos ajuda?

O **JPA (Java Persistence API)** é uma especificação do Java que define um padrão para mapear objetos Java para tabelas de um banco de dados relacional, e vice-versa. Esse mapeamento é chamado de **ORM (Object-Relational Mapping)**.

Antes do JPA, o desenvolvedor precisava escrever manualmente comandos SQL para cada operação de leitura e escrita no banco de dados. Com o JPA, descrevemos como cada classe Java se relaciona com as tabelas do banco por meio de anotações, e o framework cuida de gerar e executar o SQL automaticamente.

A implementação mais popular do JPA é o **Hibernate**, que é incluído automaticamente como implementação padrão quando usamos o `spring-boot-starter-data-jpa`.

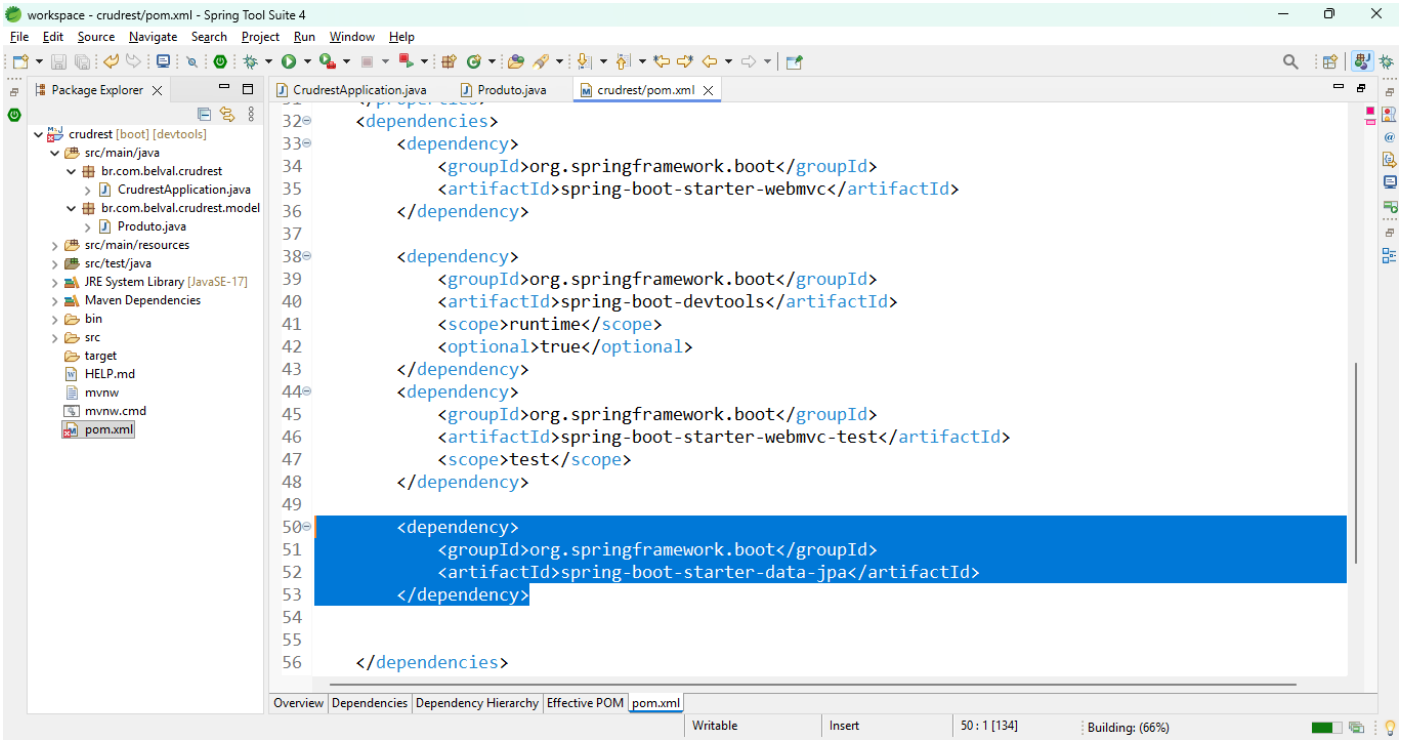
O que é o Spring Data JPA?

O **Spring Data JPA** é uma camada sobre o JPA que simplifica ainda mais o acesso a dados. Ele elimina a necessidade de escrever implementações repetitivas das operações de **CRUD (Create, Read, Update, Delete)**. Basta criar uma interface que estende `CrudRepository` e o Spring gera a implementação em tempo de execução automaticamente.

1 — Adicionando a dependência Spring Data JPA ao projeto

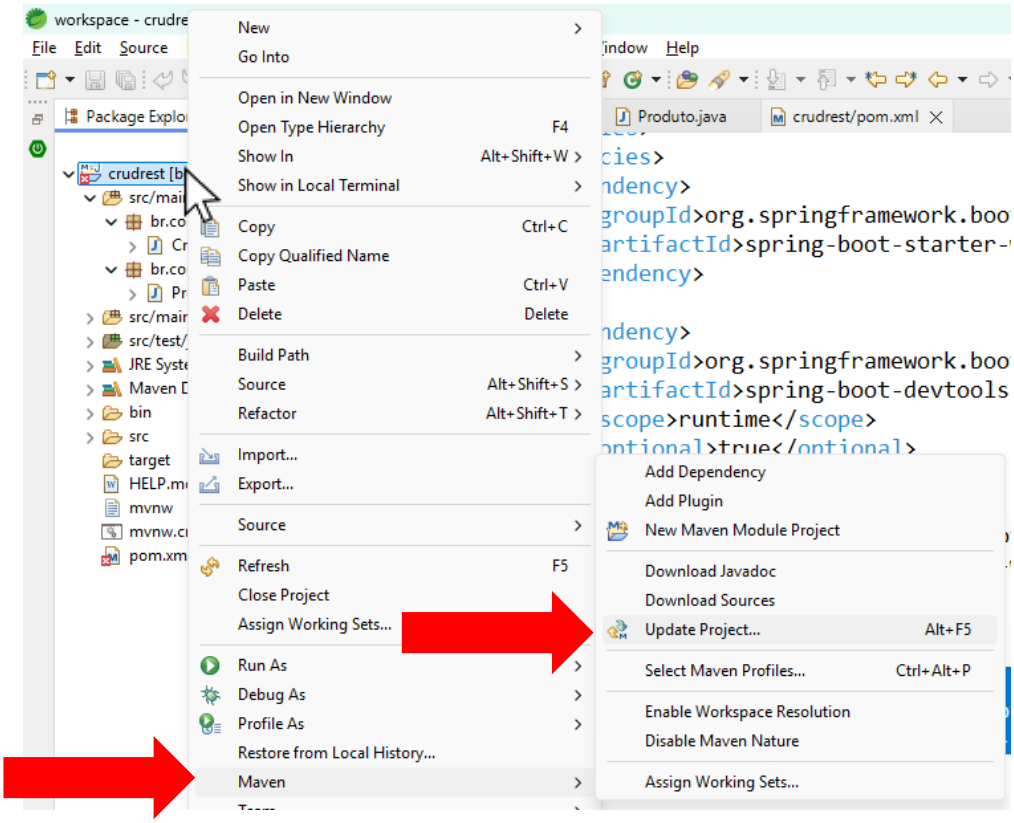
Para que o projeto passe a utilizar o Spring Data JPA, precisamos adicionar a dependência correspondente ao arquivo `pom.xml`. Abra esse arquivo, localizado na raiz do projeto, e adicione o bloco abaixo dentro da seção `<dependencies>`:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```



Essa dependência inclui automaticamente o **Hibernate** (implementação JPA), o **Spring Data JPA** e a infraestrutura necessária para conectar a aplicação ao banco de dados.

Após editar o `pom.xml`, clique com o botão direito do mouse sobre o projeto no **Package Explorer** >> **Maven** >> **Update Project** (ou pressione `ALT + F5`) para que o Eclipse baixe a nova dependência.



ATENÇÃO

O item 2 desse guia se divide em 2 caminhos, **2.a** mostra como configurar o banco de dados **H2** e o outro caminho, **2.b**, mostra como configurar nossa aplicação para funcionar com o **SQL Server**.

O arquivo `application.properties` só irá manter uma das configurações por vez, ou conterá a configuração do H2 ou do SQL Server.

Para preservarmos estas configurações, serão criados 2 arquivos:

- **`application-dev.properties`**(H2)
- **`application-prod.properties`**(SQL Server)

Quando quiser utilizar um dos bancos de dados, basta substituir o conteúdo de **`application.properties`** pelo conteúdo de um desses arquivos.

ATENÇÃO

Não adicione caracteres acentuados nos arquivos **`application.properties`**, **`application-dev.properties`** e **`application-prod.properties`**.

Nada de **ã, é, ç,** caso contrário a aplicação não funciona e aparece uma mensagem de erro que não deixa claro que esse é o problema.

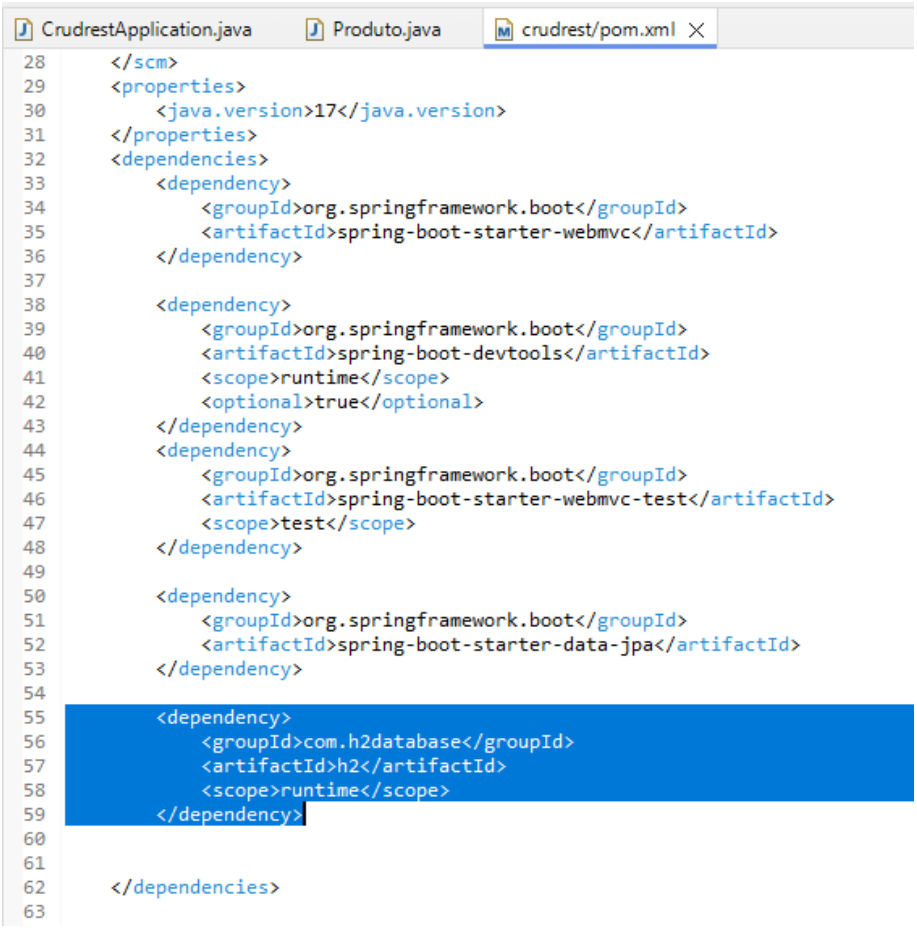
2.a – Configurando o banco de dados H2 (ambiente de desenvolvimento)

O **H2** é um banco de dados relacional escrito em Java que funciona inteiramente em memória ou como um arquivo local. Ele é ideal para o ambiente de desenvolvimento e testes por não exigir instalação de nenhum servidor externo e por ser reiniciado a cada execução da aplicação, garantindo um ambiente limpo e previsível.

2.a.1 – Adicionando a dependência do H2

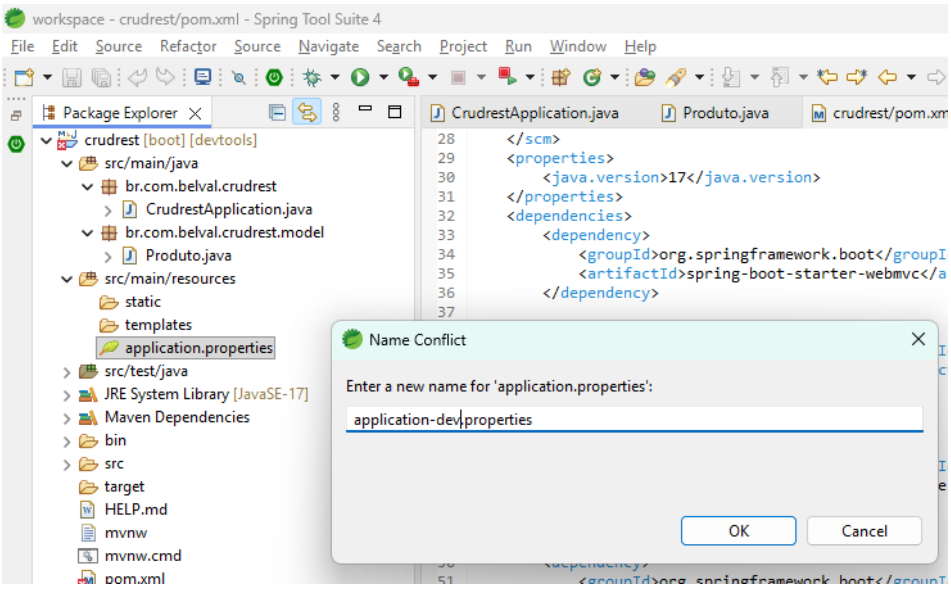
Adicione também ao `pom.xml` a dependência do banco H2, marcada com `scope` igual a `runtime`, pois ela só é necessária em tempo de execução, não em compilação:

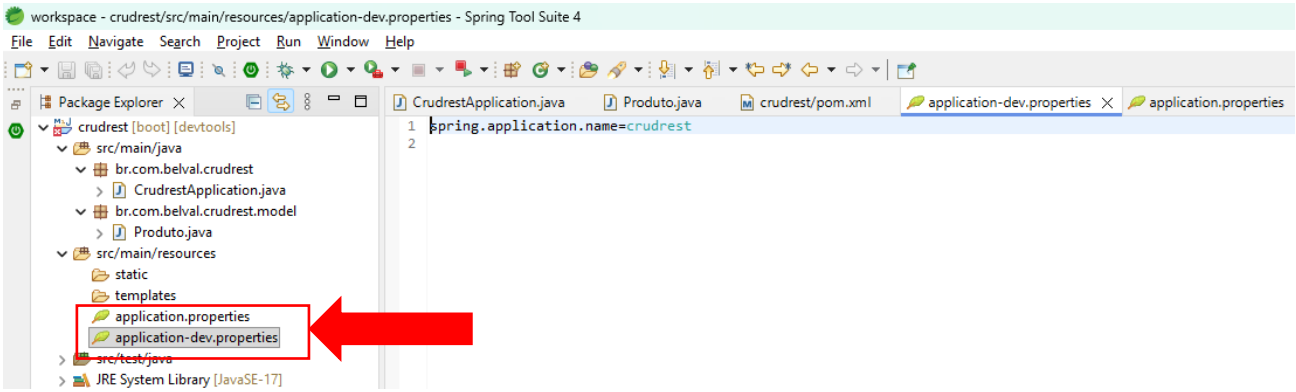
```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```



2.a.2 – Crie um arquivo application-dev.properties para o H2

Copie o arquivo `application.properties` presente em `src/main/resources` e cole no mesmo diretório com o nome de `application-dev.properties`.





2.a.3 – Configurando o application-dev.properties para o H2

Abra o arquivo `src/main/resources/application-dev.properties` e adicione as seguintes propriedades:

```
spring.application.name=crudrest

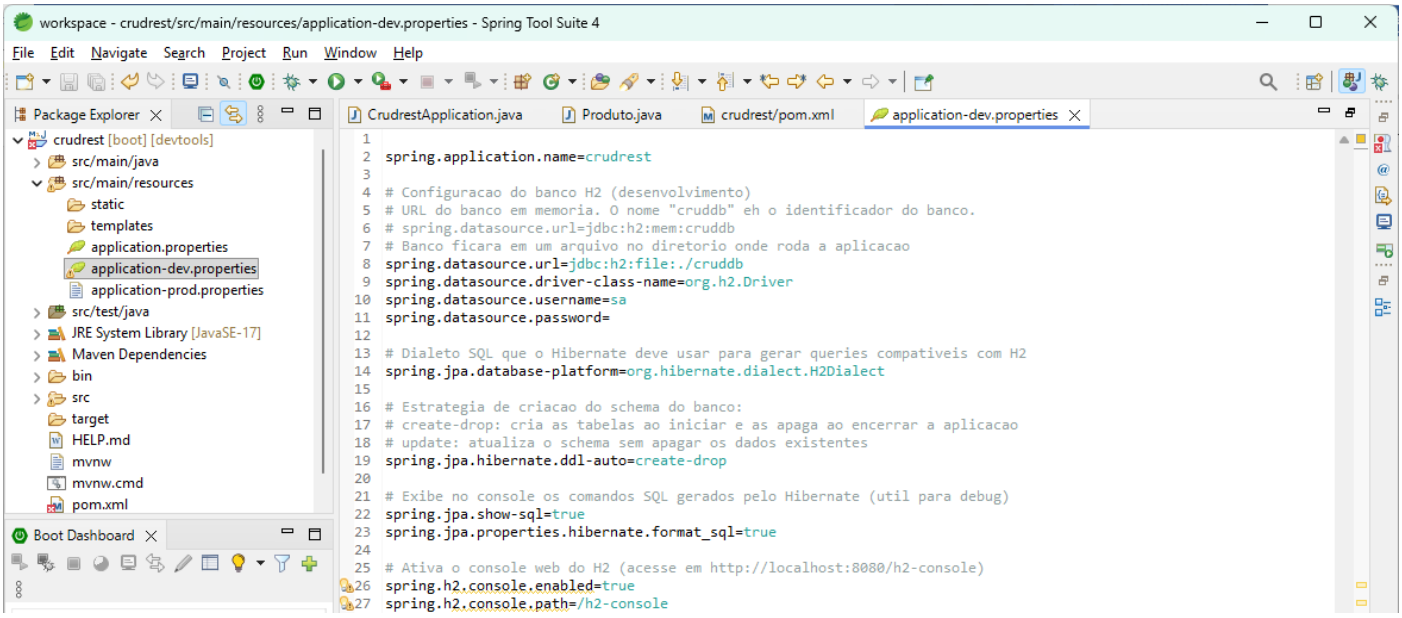
# Configuracao do banco H2 (desenvolvimento)
# URL do banco em memoria. O nome "cruddb" eh o identificador do banco.
# spring.datasource.url=jdbc:h2:mem:cruddb
# Banco ficara em um arquivo no diretorio onde roda a aplicacao
spring.datasource.url=jdbc:h2:file:./cruddb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Dialeto SQL que o Hibernate deve usar para gerar queries compativeis com H2
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Estrategia de criacao do schema do banco:
# create-drop: cria as tabelas ao iniciar e as apaga ao encerrar a aplicacao
# update: atualiza o schema sem apagar os dados existentes
spring.jpa.hibernate.ddl-auto=create-drop

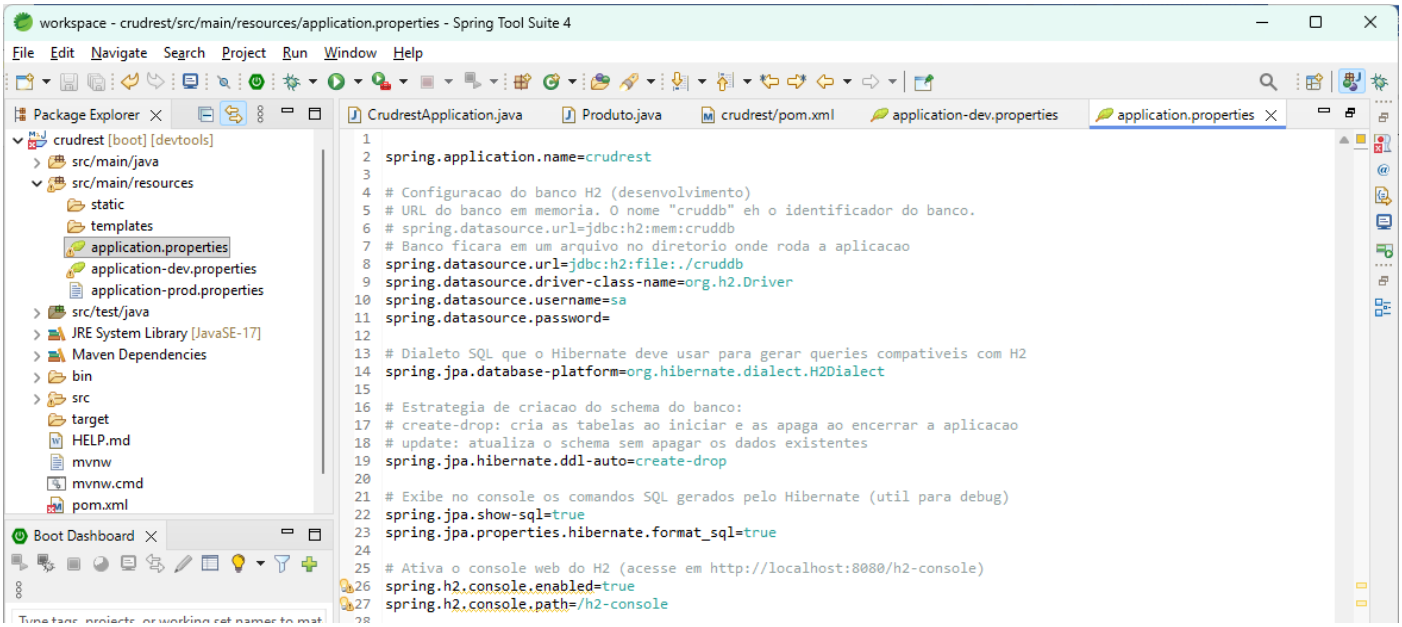
# Exibe no console os comandos SQL gerados pelo Hibernate (util para debug)
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Ativa o console web do H2 (acesse em http://localhost:8080/h2-console)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```



Com essas configurações, ao iniciar a aplicação, o Spring Boot criará automaticamente as tabelas com base nas classes anotadas com `@Entity`. O console web do H2 estará acessível no endereço `http://localhost:8080/h2-console`, onde é possível executar consultas SQL diretamente no banco durante o desenvolvimento.

Se você for utilizar o banco de dados **H2** basta copiar o conteúdo de `application-dev.properties` em `application.properties`.



2.b – Configurando o SQL Server (ambiente de produção)

Para o ambiente de produção, utilizaremos o banco de dados **Microsoft SQL Server**.

2.b.1 – Adicionando o driver JDBC do SQL Server

Adicione ao `pom.xml` a dependência do driver JDBC da Microsoft:

```
<dependency>
    <groupId>com.microsoft.sqlserver</groupId>
    <artifactId>mssql-jdbc</artifactId>
    <scope>runtime</scope>
</dependency>
```

```xml
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>

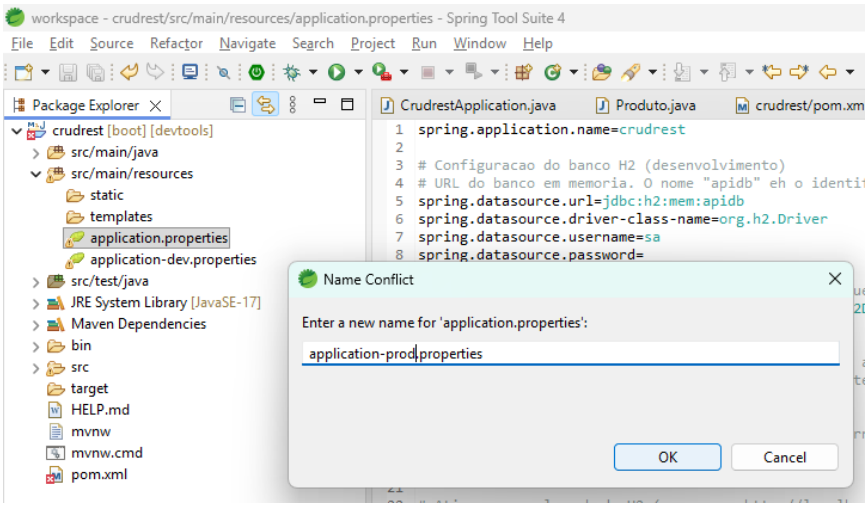
        <dependency>
            <groupId>com.microsoft.sqlserver</groupId>
            <artifactId>mssql-jdbc</artifactId>
            <scope>runtime</scope>
        </dependency>

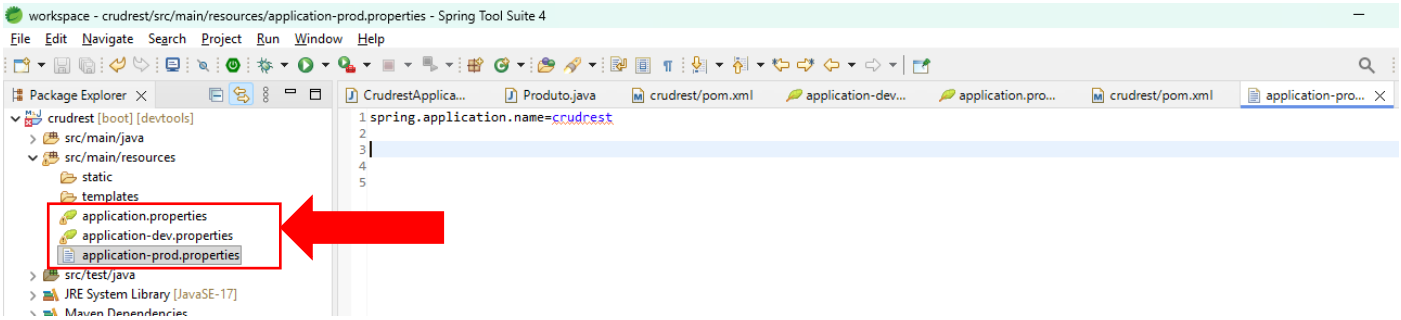
    </dependencies>
```

O Spring Boot gerencia automaticamente a versão compatível do `mssql-jdbc` quando o projeto usa o Spring Boot BOM (*Bill of Materials*), portanto não é necessário informar a versão diretamente.

2.b.2 – Crie um arquivo application-prod.properties para o SQL Server

Copie o arquivo `application.properties` presente em `src/main/resources` e cole no mesmo diretório com o nome de `application-prod.properties`.





2.b.3 – Configurando o application-prod.properties para o SQL Server

Substitua (ou crie em um arquivo separado para produção) as configurações do banco de dados pelas propriedades abaixo, ajustando o servidor, a porta, o nome do banco, o usuário e a senha conforme o ambiente:

```
spring.application.name=crudrest

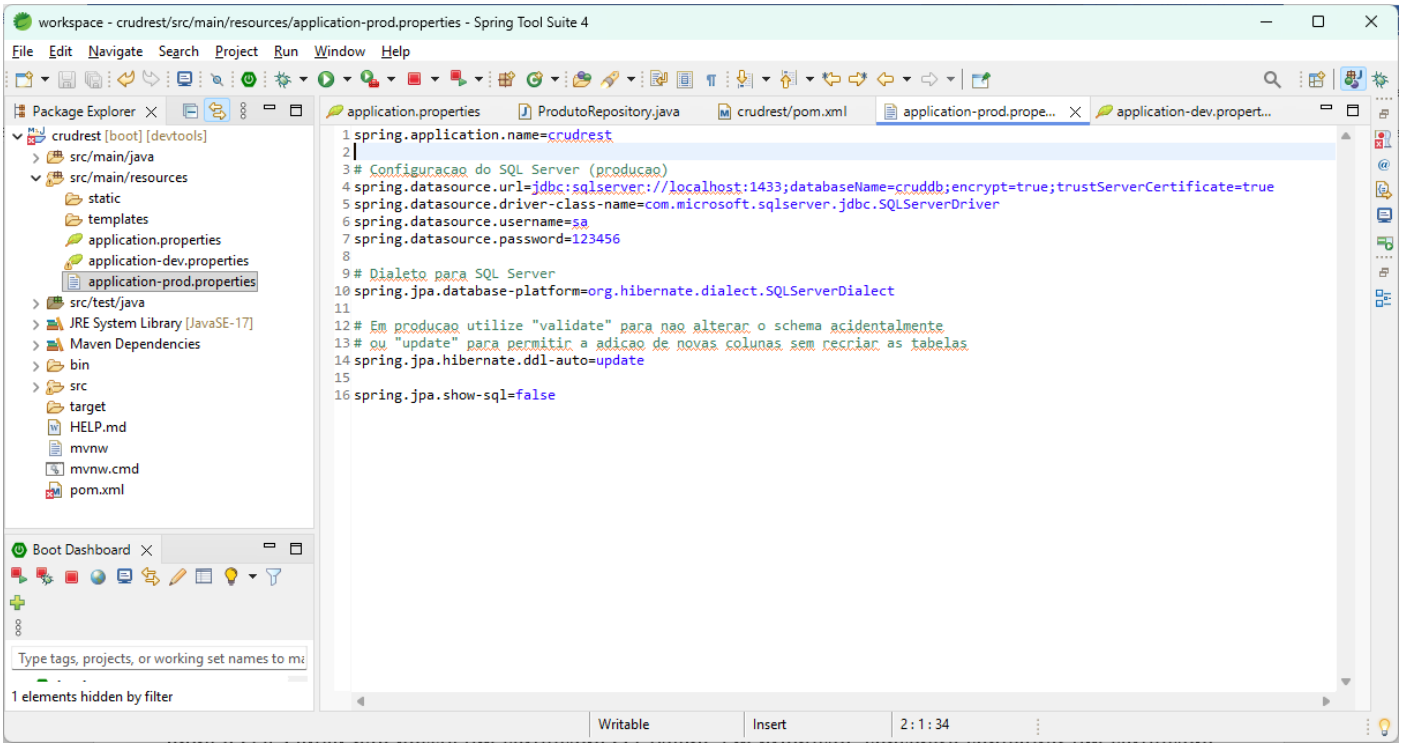
# Configuracao do SQL Server (producao)
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=cruddb;encrypt=tr
ue;trustServerCertificate=true

spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
spring.datasource.username=sa
spring.datasource.password=123456

# Dialeto para SQL Server
spring.jpa.database-platform=org.hibernate.dialect.SQLServerDialect

# Em producao utilize "validate" para nao alterar o schema acidentalmente
# ou "update" para permitir a adicao de novas colunas sem recriar as tabelas
spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=false
```



O parâmetro `trustServerCertificate=true` é necessário em ambientes locais de desenvolvimento onde o SQL Server não possui um certificado SSL válido. Em produção, considere configurar um certificado adequado e remover esse parâmetro.

3 — Anotando a classe Produto com JPA

Para que o JPA reconheça a classe `Produto` como uma entidade persistente e saiba como mapeá-la para a tabela do banco de dados, precisamos adicionar um conjunto de anotações do pacote `jakarta.persistence` (ou `javax.persistence` em versões mais antigas do Spring Boot).

A seguir veremos o resultado final da classe anotada e a explicação de cada anotação.

Resultado final da classe Produto anotada

```java
package br.com.belval.crudrest.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
```

```java
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Lob;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.Objects;

@Entity
@Table(name = "produto")
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

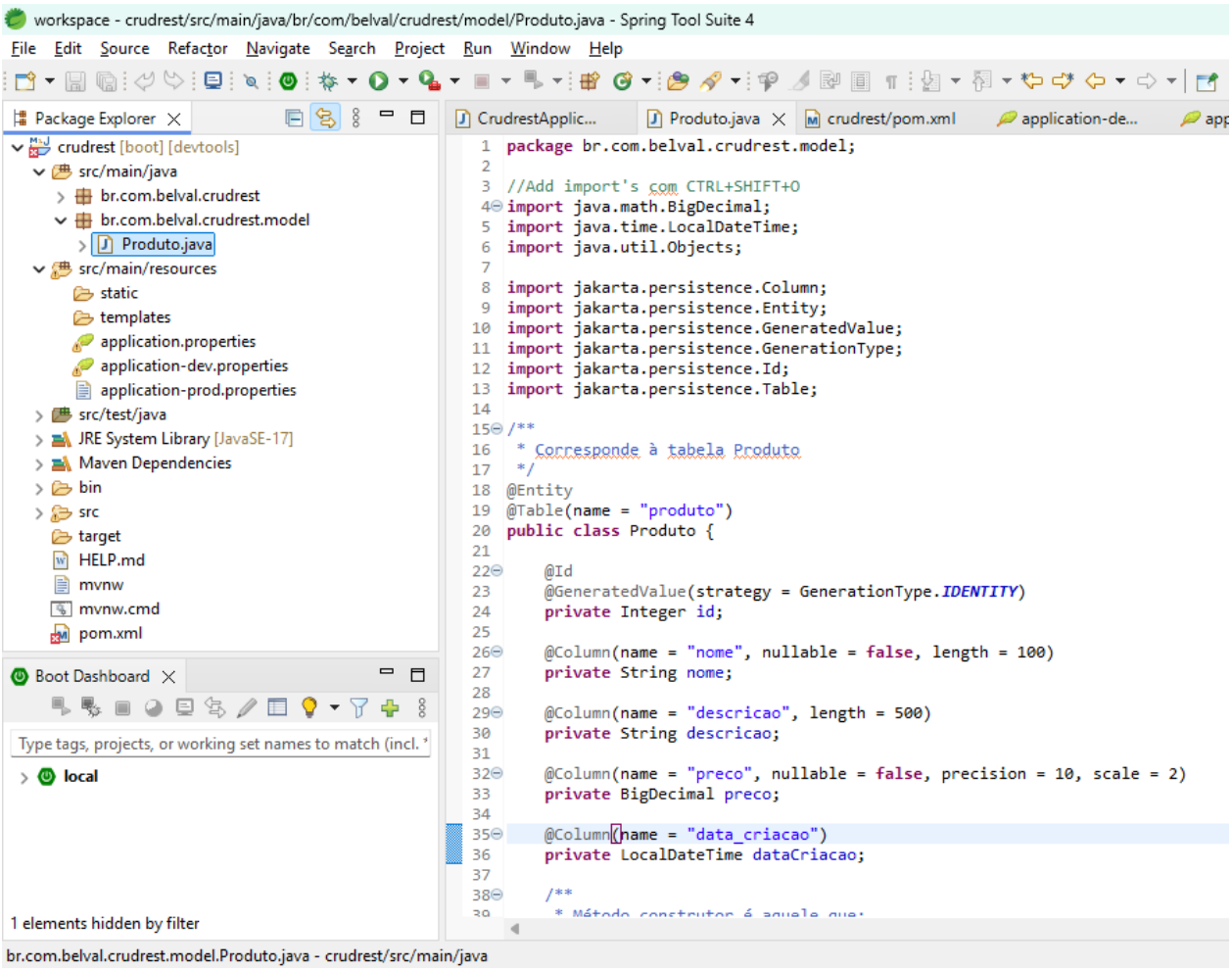
    @Column(name = "nome", nullable = false, length = 100)
    private String nome;

    @Column(name = "descricao", length = 500)
    private String descricao;

    @Column(name = "preco", nullable = false, precision = 10, scale = 2)
    private BigDecimal preco;

    @Column(name = "data_criacao")
    private LocalDateTime dataCriacao;

    // ... construtores, getters, setters, equals, hashCode e toString
}
```



Entendendo cada anotação

@Entity

A anotação `@Entity` informa ao JPA que esta classe representa uma entidade persistente, isto é, que seus objetos devem ser armazenados em uma tabela do banco de dados. Toda classe anotada com `@Entity` obrigatoriamente precisa ter um construtor padrão (sem parâmetros) público ou protegido.

@Table(name = "produto")

A anotação `@Table` especifica o nome da tabela no banco de dados à qual esta entidade está mapeada. Se omitida, o JPA usa o nome da própria classe como nome da tabela. É uma boa prática informá-la explicitamente para evitar ambiguidades entre convenções de nomenclatura do Java (*camelCase*) e do SQL (*snake_case*).

@Id

Marca o atributo que será a chave primária da tabela. Toda entidade JPA deve ter exatamente um atributo anotado com `@Id`.

@GeneratedValue(strategy = GenerationType.IDENTITY)

Define a estratégia de geração automática do valor da chave primária. A estratégia `IDENTITY` delega a geração do ID ao próprio banco de dados, utilizando uma coluna de auto-incremento (como `IDENTITY` no SQL Server ou `AUTO_INCREMENT` no MySQL). Outras estratégias disponíveis são:

- **SEQUENCE**: utiliza uma sequence do banco (recomendado para PostgreSQL e Oracle).
- **TABLE**: utiliza uma tabela auxiliar para controlar o próximo valor (compatível com qualquer banco, porém mais lento).
- **AUTO**: o Hibernate escolhe automaticamente a melhor estratégia para o banco configurado.

@Column

Mapeia o atributo para uma coluna específica da tabela. Os principais atributos dessa anotação são:

- `name`: define o nome da coluna no banco. Se omitido, usa o nome do atributo Java.
- `nullable`: quando `false`, define a coluna como `NOT NULL` no DDL gerado pelo Hibernate.
- `length`: define o tamanho máximo da coluna para campos do tipo `VARCHAR`.
- `precision` e `scale`: usados em campos `BigDecimal`, `precision` é o número total de dígitos e `scale` é o número de dígitos após o ponto decimal. Por exemplo, `precision = 10, scale = 2` permite valores como `99999999.99`.

Mapeamento de tipos de dados Java ↔ SQL

O JPA realiza a conversão entre os tipos do Java e os tipos de colunas do banco de dados automaticamente. A tabela a seguir apresenta as correspondências mais comuns, incluindo tipos além dos presentes na classe Produto:

Tipo Java	Tipo SQL	Observações
`String`	`VARCHAR(255)`	Tamanho padrão 255. Use `@Column(length=N)` para alterar.
`Integer / int`	`INTEGER / INT`	*Wrapper* `Integer` é preferível para permitir valor nulo.
`Long / long`	`BIGINT`	Ideal para IDs em tabelas com muitos registros.
`BigDecimal`	`DECIMAL(p,s)`	Use precision e scale via `@Column` para controle exato.

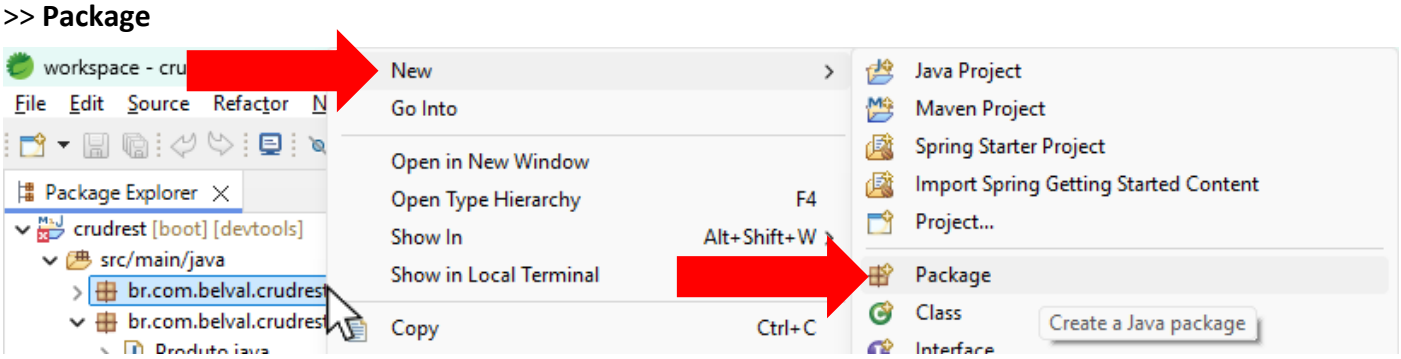
`Double / double`	`DOUBLE / FLOAT`	Evite para valores monetários; prefira `BigDecimal`.
`Boolean / boolean`	`BOOLEAN / BIT`	Armazenado como 0 ou 1 em alguns bancos.
`LocalDate`	`DATE`	Apenas data: dia, mês, ano.
`LocalDateTime`	`TIMESTAMP`	Data e hora completa, sem fuso horário.
`LocalTime`	`TIME`	Apenas hora, minuto e segundo.
`OffsetDateTime`	`TIMESTAMP WITH TIMEZONE`	Data e hora com informação de fuso horário.
`byte[]`	`BLOB / VARBINARY`	Use com `@Lob` para arquivos e imagens.
`String (texto longo)`	`CLOB / TEXT / NVARCHAR(MAX)`	Use com `@Lob` para textos muito longos.
`Enum`	`VARCHAR ou INTEGER`	Use `@Enumerated` para controlar o mapeamento.
`UUID`	`UNIQUEIDENTIFIER / CHAR(36)`	Útil como chave primária alternativa.

4 — Criando a interface ProdutoRepository

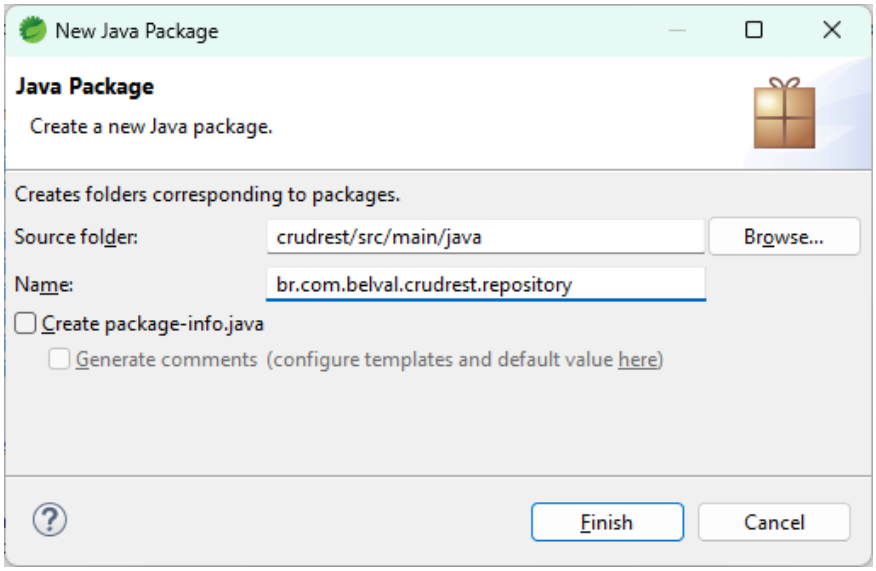
No padrão de projeto **Repository**, criamos uma interface dedicada à comunicação com o banco de dados para cada entidade. Com o Spring Data JPA, basta criar uma interface que estenda `CrudRepository<T, ID>`, onde `T` é o tipo da entidade e `ID` é o tipo da chave primária. O Spring gera automaticamente toda a implementação em tempo de execução.

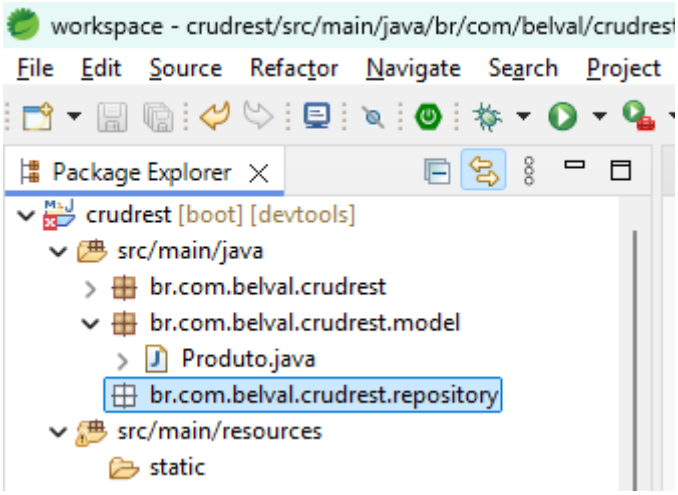
Criando o pacote repository

4.1 — No **Package Explorer**, clique com o botão direito sobre o pacote `br.com.belval.crudrest` >> **New** >> **Package**



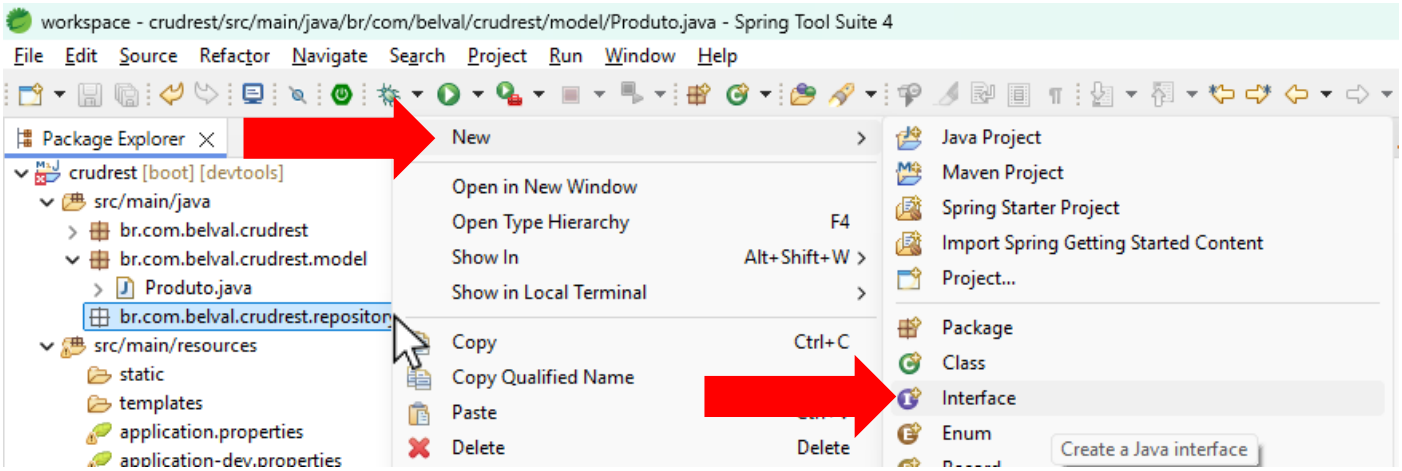
4.2 — No campo "Name", informe `br.com.belval.crudrest.repository` e clique em "Finish"



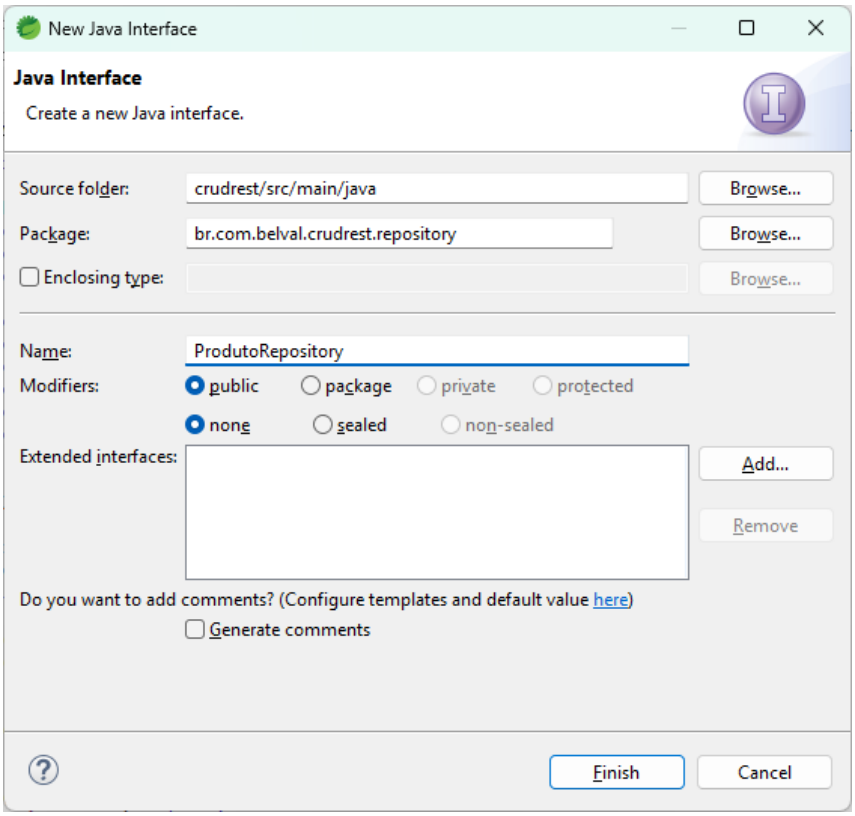


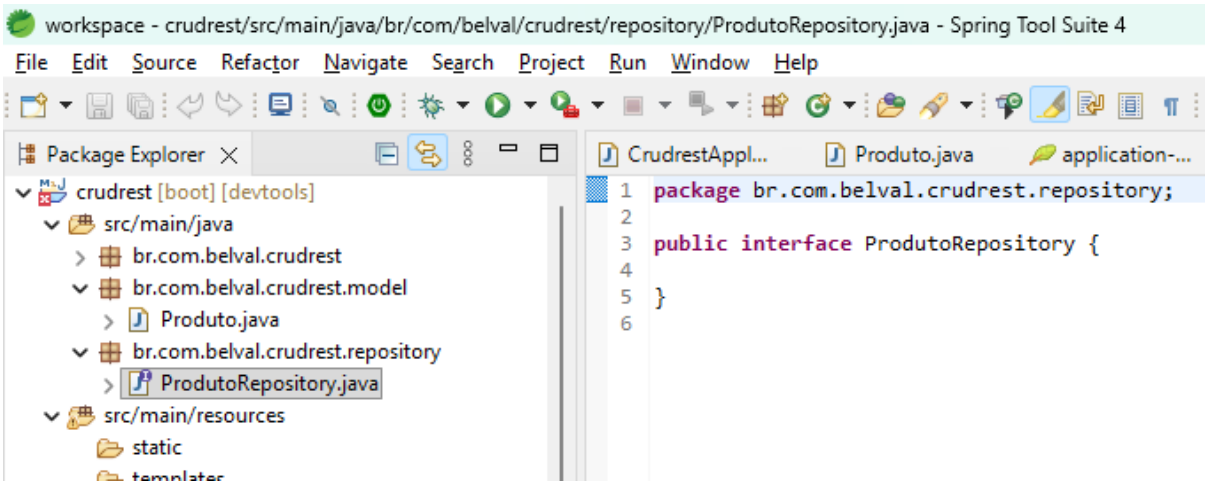
Criando a interface ProdutoRepository

4.3 — Clique com o botão direito sobre o pacote `br.com.belval.crudrest.repository` >> **New** >> **Interface**



4.4 — No campo "Name", informe `ProdutoRepository` e clique em "Finish"





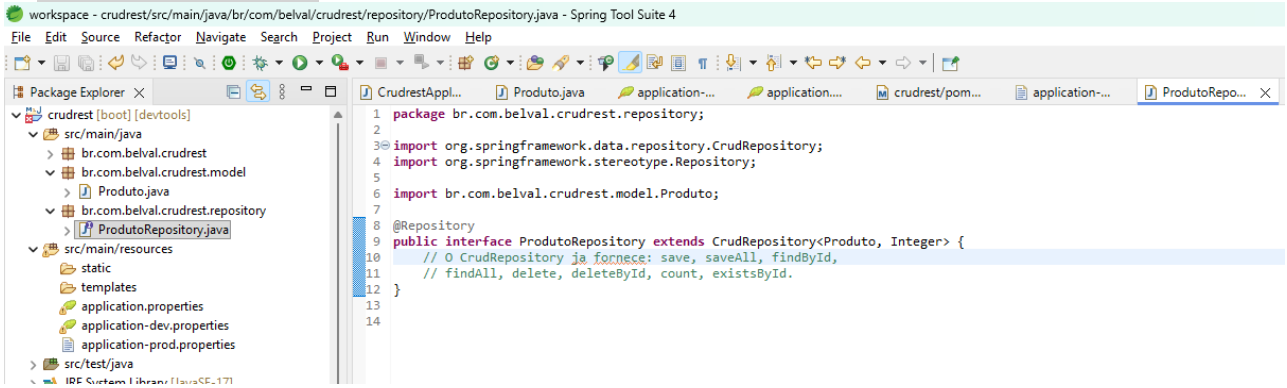
4.5 — Edite a interface gerada para que fique conforme o código abaixo:

```java
package br.com.belval.crudrest.repository;

import br.com.belval.crudrest.model.Produto;
import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ProdutoRepository extends CrudRepository<Produto, Integer> {
    // O CrudRepository ja fornece: save, saveAll, findById,
    // findAll, delete, deleteById, count, existsById.
}
```

Se `@Repository` ou `CrudRepository` apresentarem erro, não esqueça de salvar as alterações, utilizar o atalho `CTRL + SHIFT + O` para adicionar os *import's* e salvar o arquivo novamente.



O que o CrudRepository nos oferece?

Ao estender `CrudRepository<Produto, Integer>`, a interface `ProdutoRepository` herda automaticamente os seguintes métodos, sem necessidade de escrevê-los:

Método	O que faz
`save(produto)`	Insere ou atualiza um Produto no banco.
`findById(id)`	Retorna Optional<Produto> pelo ID.
`findAll()`	Retorna uma List<Produto> com todos os registros.
`deleteById(id)`	Remove o Produto com o ID informado.
`count()`	Retorna o número total de registros.
`existsById(id)`	Retorna true se existir um Produto com aquele ID.

Criando consultas personalizadas com Query Methods

Além dos métodos herdados, o Spring Data JPA permite criar consultas personalizadas apenas declarando métodos na interface seguindo uma convenção de nomenclatura. O Spring interpreta o nome do método e gera automaticamente o SQL correspondente.

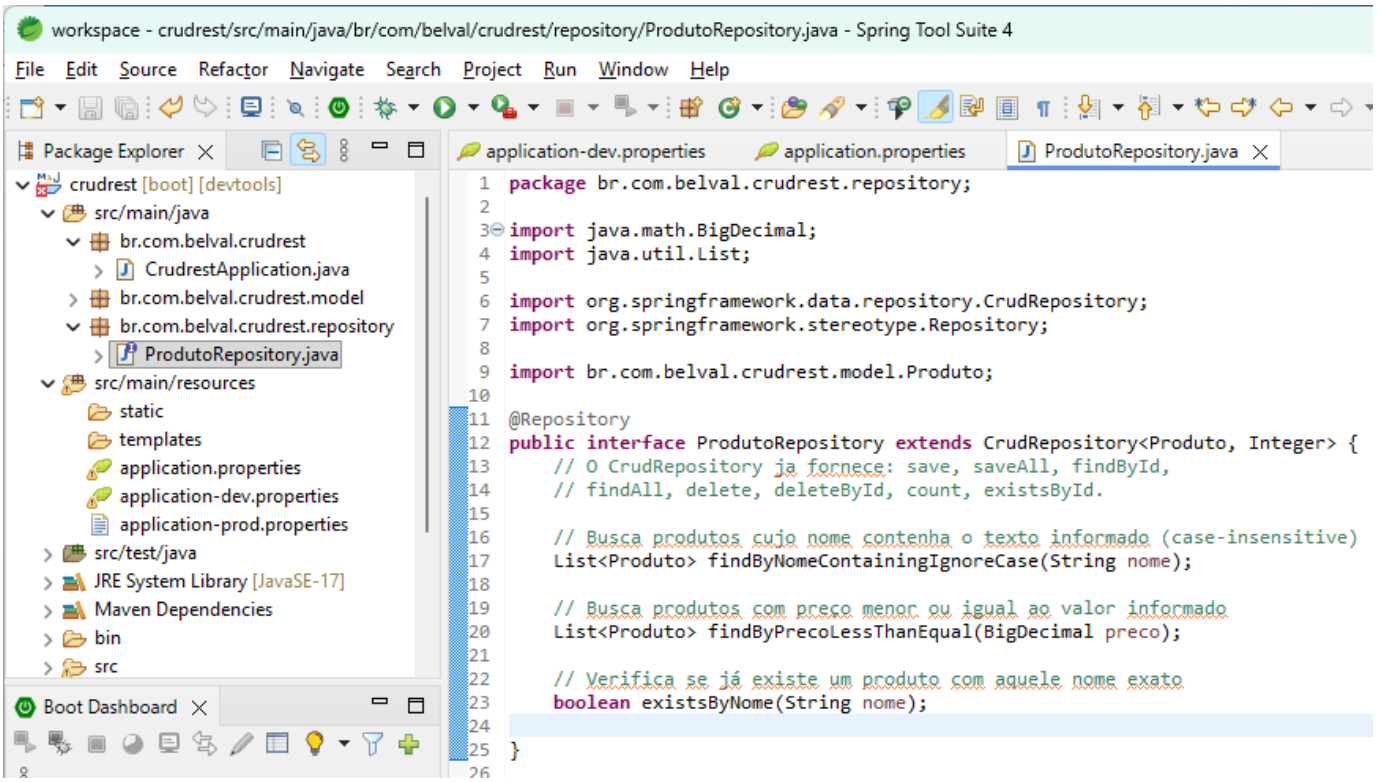
4.6 – Adicione as assinaturas dos métodos `findByNomeContainingIgnoreCase`, `findByPrecoLessThanEqual` e `existsByNome`

```java
@Repository
public interface ProdutoRepository extends CrudRepository<Produto, Integer> {
    // O CrudRepository ja fornece: save, saveAll, findById,
    // findAll, delete, deleteById, count, existsById.

    // Busca produtos cujo nome contenha o texto informado (case-insensitive)
    List<Produto> findByNomeContainingIgnoreCase(String nome);

    // Busca produtos com preço menor ou igual ao valor informado
    List<Produto> findByPrecoLessThanEqual(BigDecimal preco);

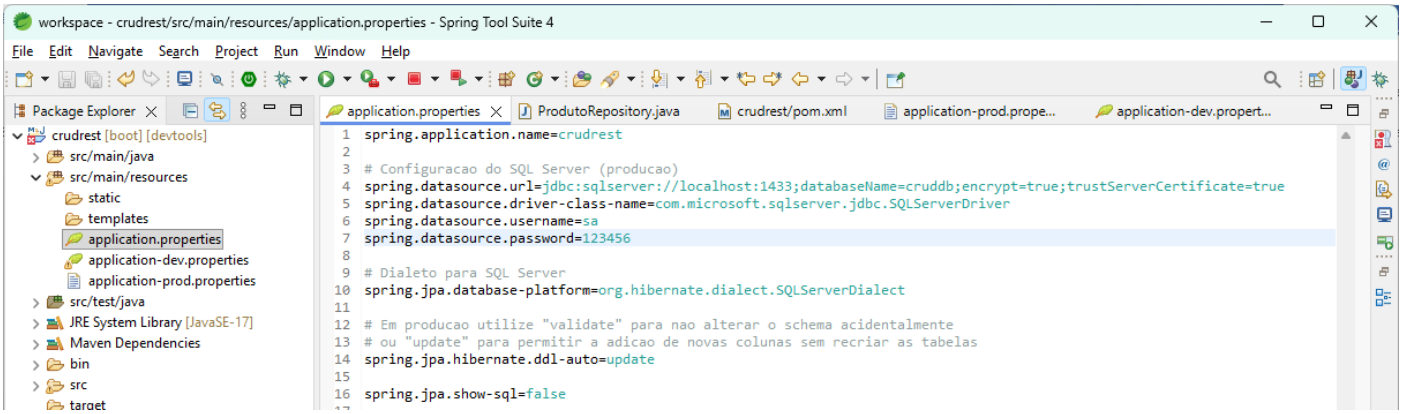
    // Verifica se já existe um produto com aquele nome exato
    boolean existsByNome(String nome);
}
```



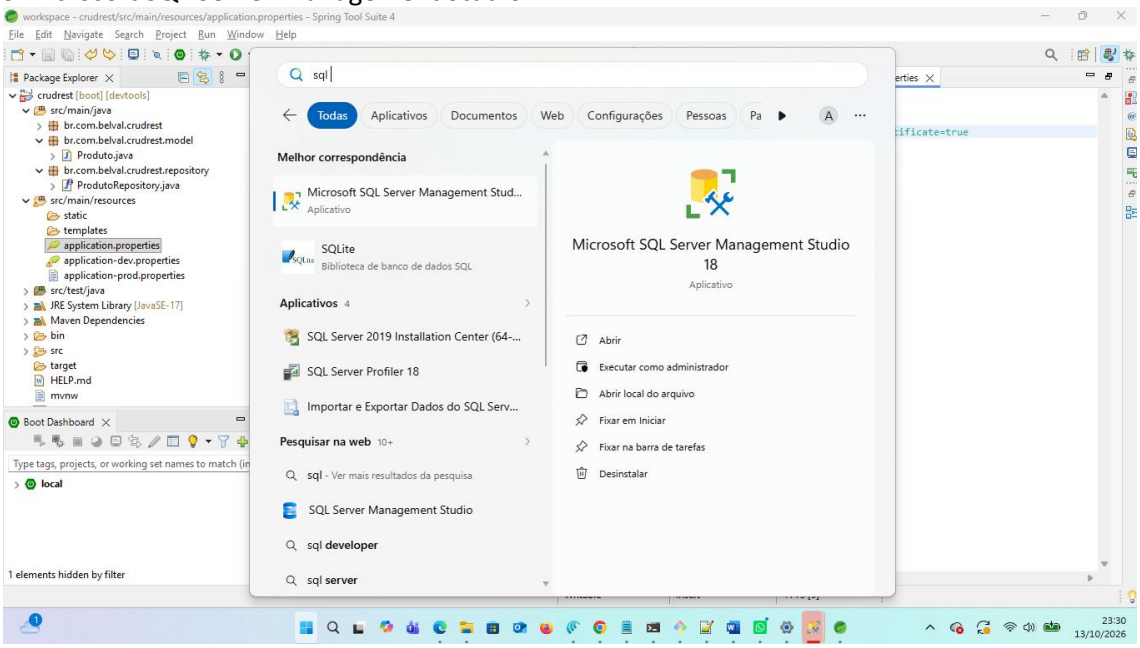
O Spring analisa o nome do método e constrói a query correspondente. O prefixo `findBy` indica uma busca, seguido do nome do atributo (`Nome`, `Preco`) e de uma condição opcional (`Containing`, `LessThanEqual`, `IgnoreCase`). Essa abordagem é chamada de **Derived Query Methods**.

5 — Execute o projeto

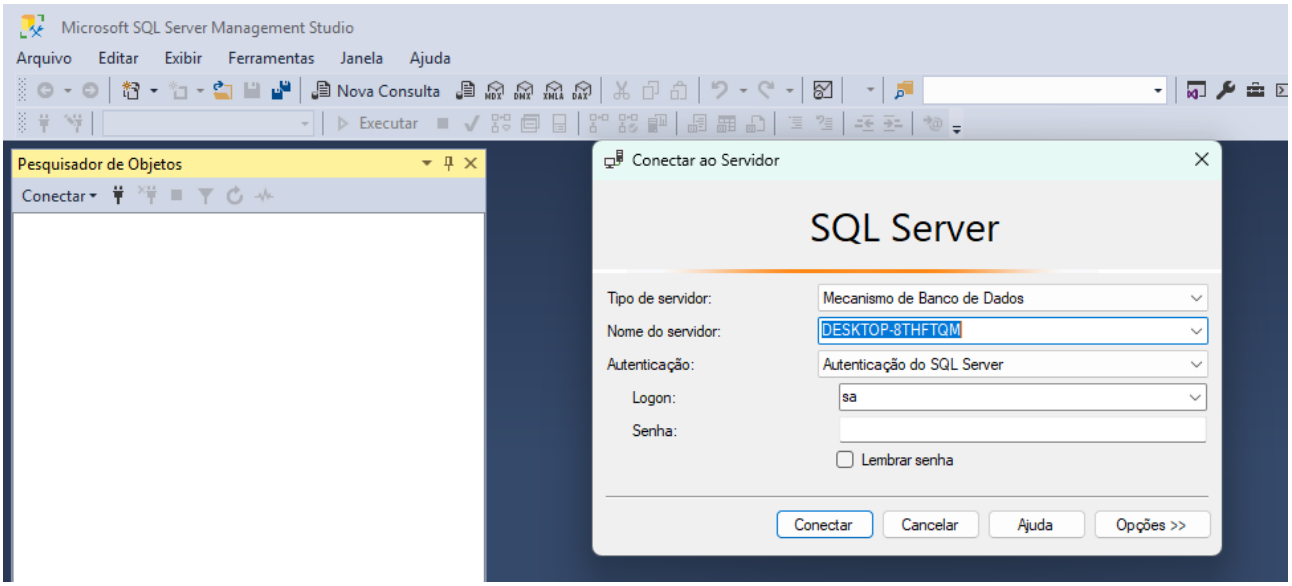
5.1 – Escolha um banco de dados, aqui vamos escolher o **SQL Server**, e vamos copiar o conteúdo de `application-prod.properties` em `application.properties`.



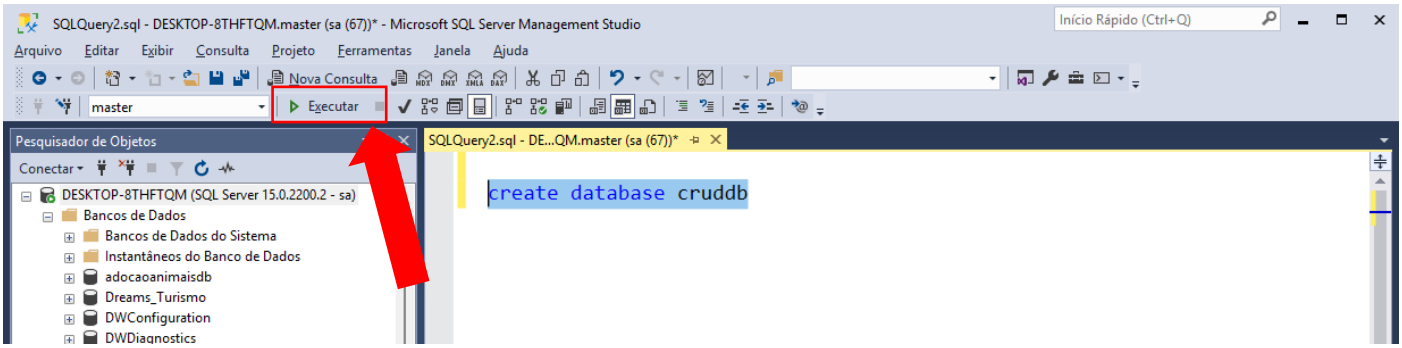
5.2 – Abra o Microsoft SQL Server Management Studio



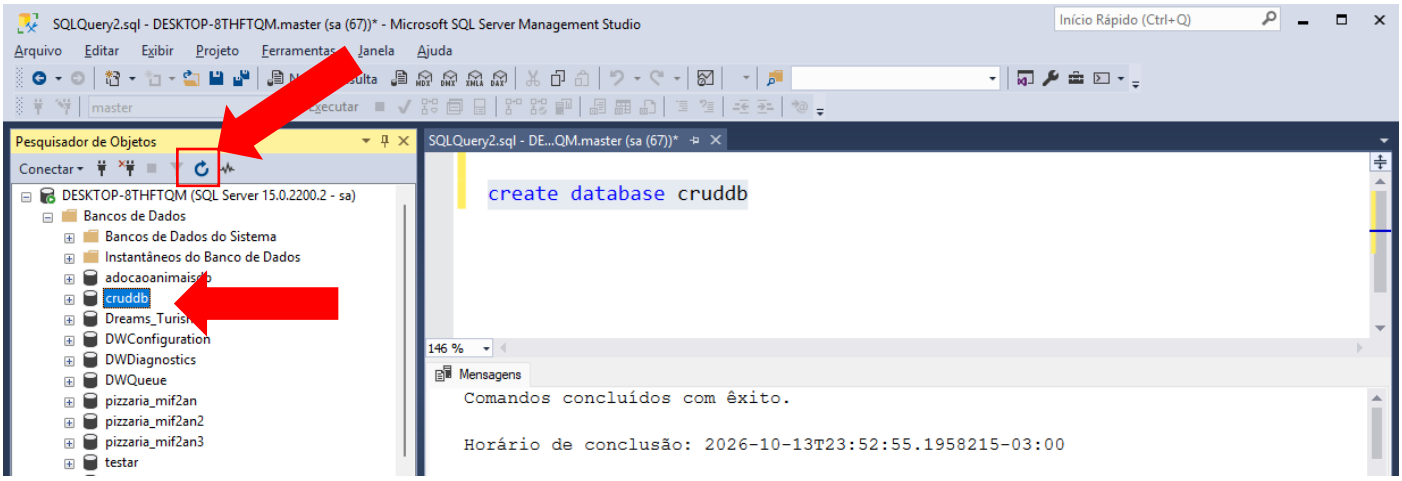
5.3 – Informe usuário e senha para se conectar aos SGBD, no laboratório da escola o usuário é `sa` e a senha é `@ITB123456`



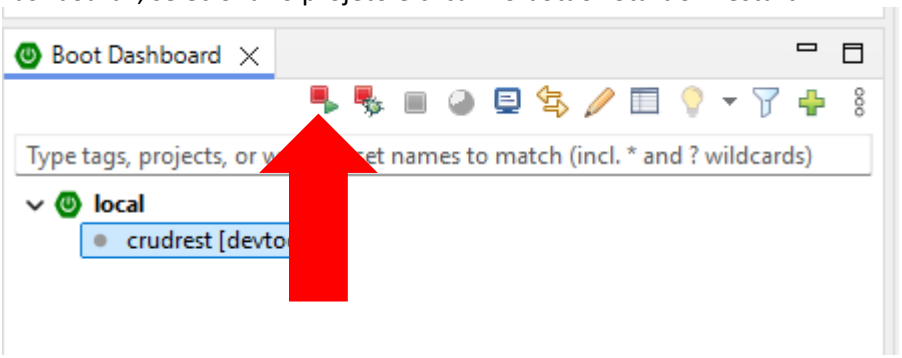
5.4 – No SQL Server, crie o database `cruddb`, digitando o comando `create database cruddb` e clicando em "Executar".

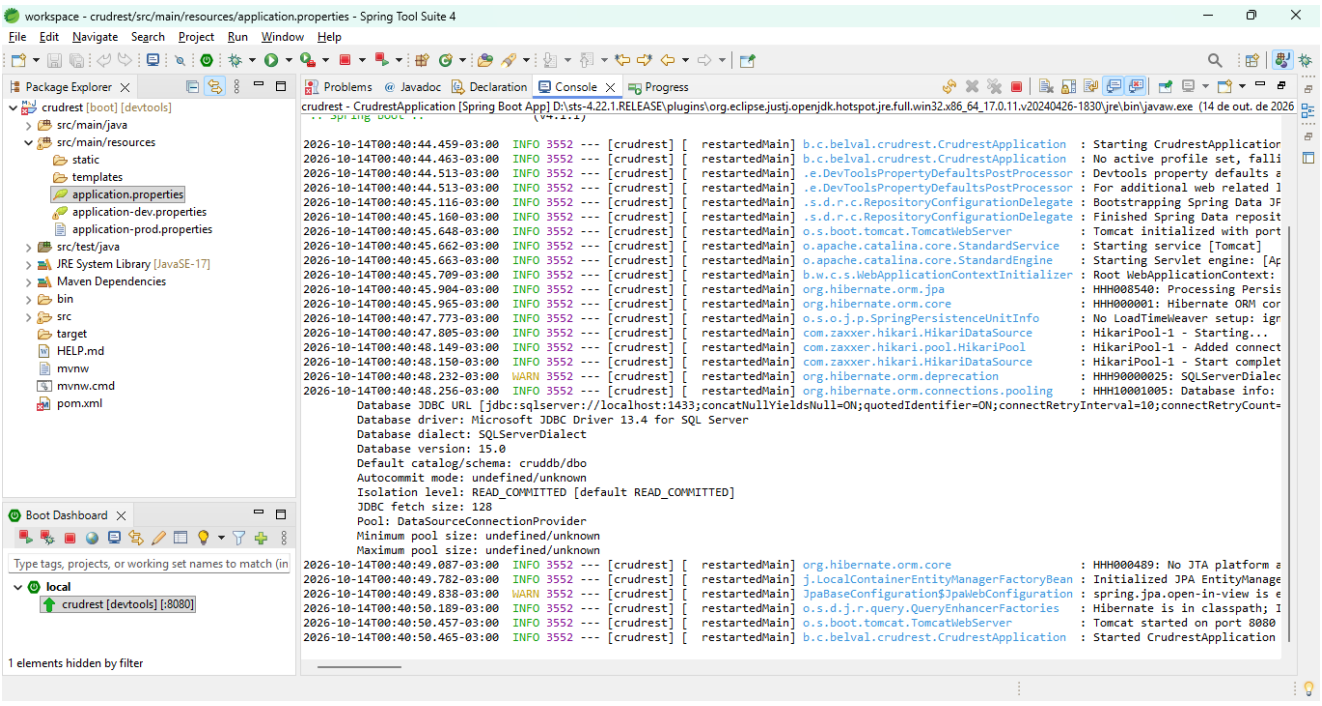


5.5 – Clique no botão de *refresh* para ver no novo *database* criado



5.6 – Na view “Boot Dashboard”, selecionar o projeto e clicar no botão “Start or Restart”.

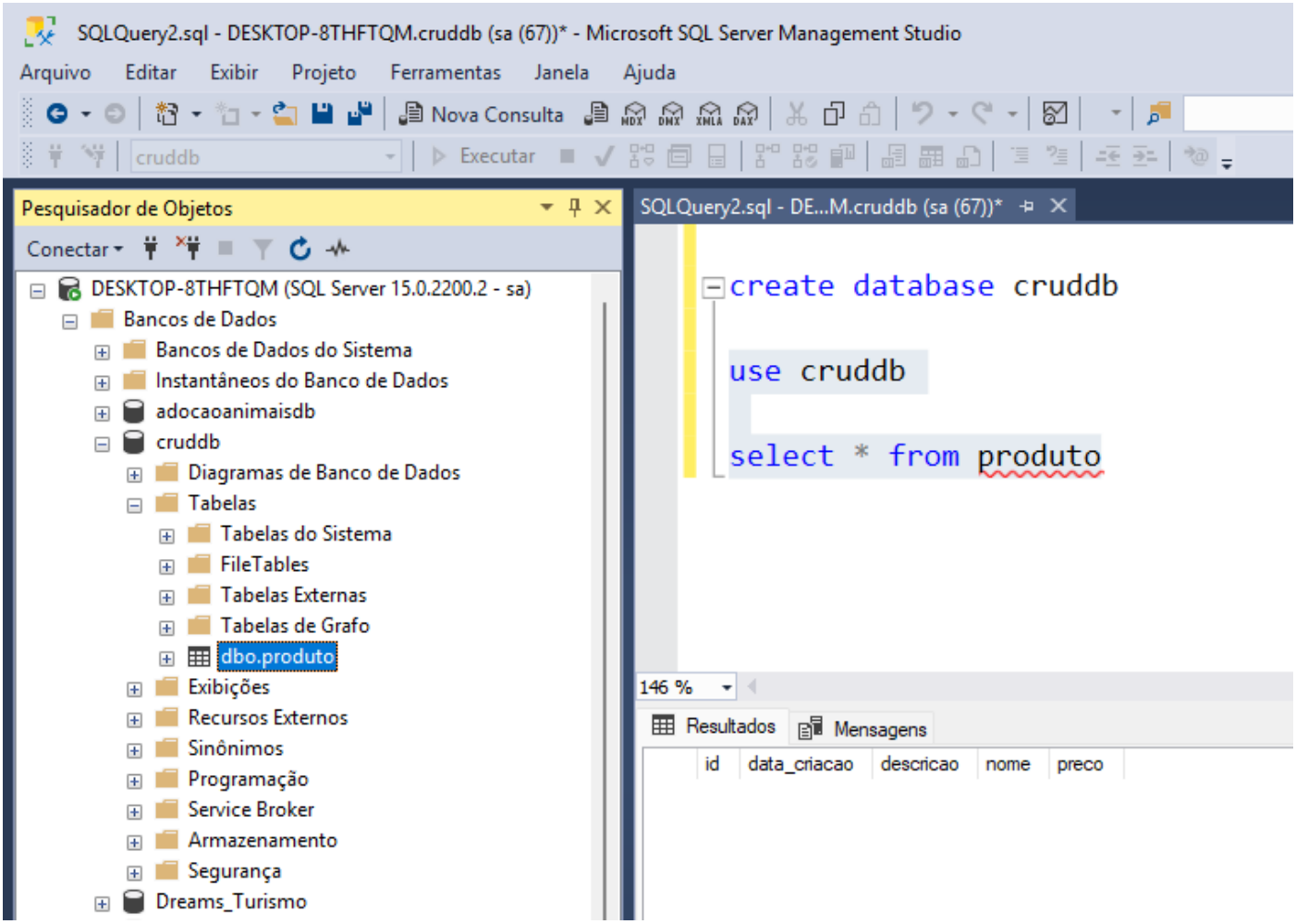




5.7 – No Microsoft SQL Server Management Studio, execute os seguintes comandos:

```
use cruddb

select * from produto
```



6 — Commitando as alterações no Git

Após realizar todas as alterações, salve os arquivos e execute os comandos abaixo no Git Bash para registrar as mudanças no repositório:

```
git status
git add .
git status
git commit -m "Add configuração do JPA incluindo a conexão com o banco e a classe Produto
recebeu as anotações de entidade"
git push origin main
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   pom.xml
        modified:   src/main/java/br/com/belval/crudrest/model/Produto.java
        modified:   src/main/resources/application.properties

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        src/main/java/br/com/belval/crudrest/repository/
        src/main/resources/application-dev.properties
        src/main/resources/application-prod.properties

no changes added to commit (use "git add" and/or "git commit -a")

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git add .
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/resources/application.properties', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/resources/application-dev.properties', LF will be replaced by CRLF the next time Git touches
 it

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   pom.xml
        modified:   src/main/java/br/com/belval/crudrest/model/Produto.java
        new file:   src/main/java/br/com/belval/crudrest/repository/ProdutoRepository.java
        new file:   src/main/resources/application-dev.properties
        new file:   src/main/resources/application-prod.properties
        modified:   src/main/resources/application.properties


takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git commit -m "Add configuração do JPA incluindo a conexão com o banco e a classe Produto recebeu as anotações de entidade"
[main 400cb2a] Add configuração do JPA incluindo a conexão com o banco e a classe Produto recebeu as anotações de entidade
 6 files changed, 121 insertions(+)
 create mode 100644 src/main/java/br/com/belval/crudrest/repository/ProdutoRepository.java
 create mode 100644 src/main/resources/application-dev.properties
 create mode 100644 src/main/resources/application-prod.properties
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git push origin main
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 8 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (18/18), 2.84 KiB | 969.00 KiB/s, done.
Total 18 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 3 local objects.
To https://github.com/ProfNpc/crudrest.git
   1a6f89e..400cb2a  main -> main

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```